

Hidden Oven

Role-Based Workflow, Order Lifecycle & Model Schema Design

Scope. This design translates the supplied e-commerce order-management requirements into an operational workflow and a normalized data model. The core system has a customer ordering journey, a staff Kanban pipeline, and an admin control plane for catalog, payment methods, analytics, history, and staff access.

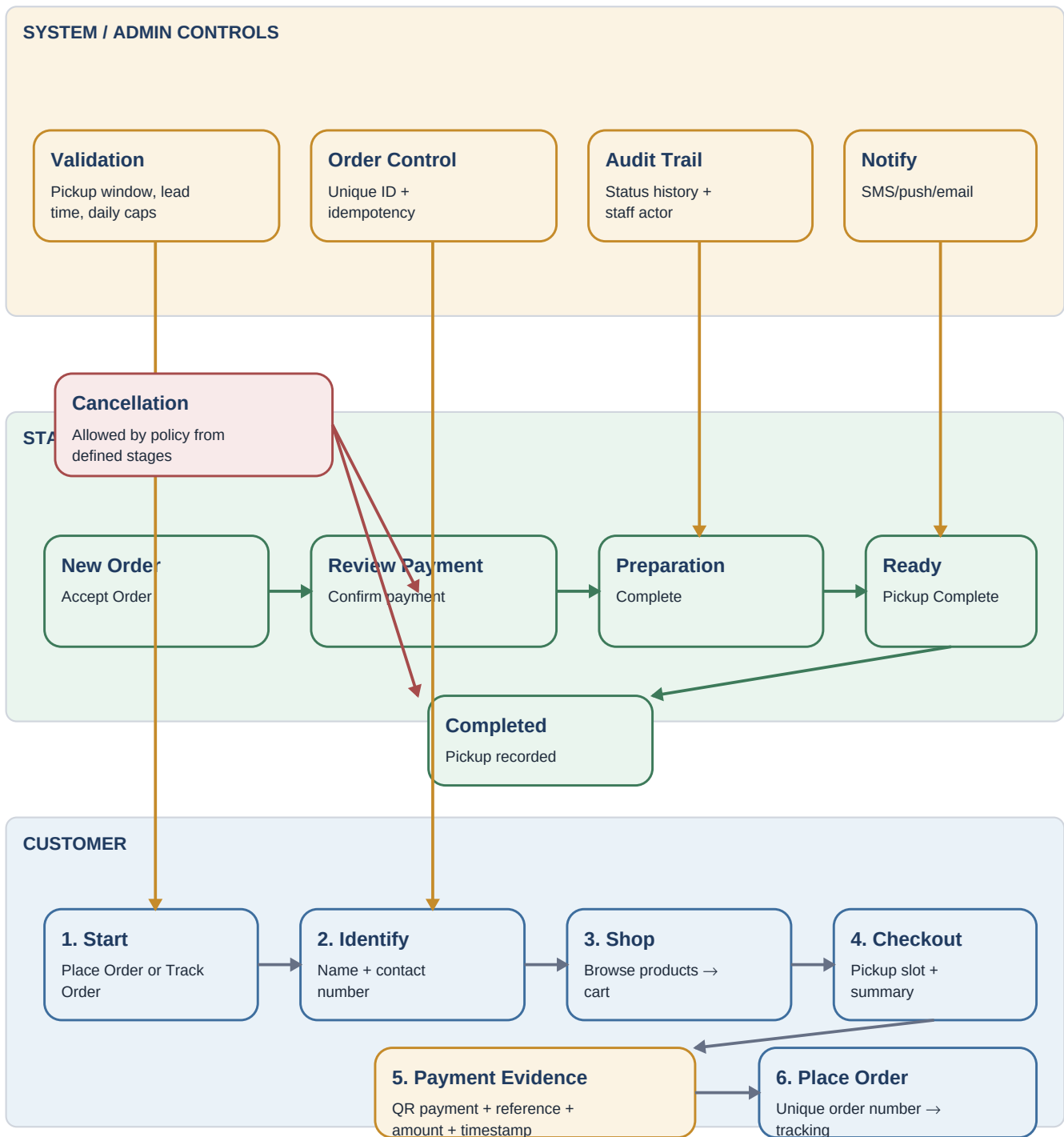
Primary actors	Primary responsibilities
Customer	Provide contact details, browse products, create a cart, select pickup time, submit payment evidence, place an order, and track status.
Staff	Accept/review orders, verify payment, prepare products, mark orders ready, and complete pickup.
Admin	Manage products, payment methods, staff access, reporting, history, and operational configuration.
System	Enforce inventory/quantity rules, generate unique order numbers, maintain status history, validate pickup windows, and trigger notifications.

Design principle: treat the order as the central aggregate, with immutable order-item snapshots, append-only status history, and explicit payment verification. This keeps the operational workflow auditable and prevents later product edits from changing historical orders.

Source reviewed: the supplied document defines the customer flow from landing page through tracking, the staff stages New Order → Review Payment → Order Preparation → Ready for Pick-up → Completed, plus a Cancelled path and an admin panel for catalog, analytics, history/logs, payment methods, and staff access. [filecite turn0file0L1-L1](#)

1. Role-Based End-to-End Flow

The following flow separates customer actions from staff actions and shows where the system owns validation, state transitions, and notifications.



Recommended state machine: NEW → PAYMENT_REVIEW → PREPARING → READY_FOR_PICKUP → COMPLETED. CANCELLED should be a controlled terminal state with a recorded reason, actor, timestamp, and refund outcome where applicable.

2. Role & Permission Matrix

Capability	Customer	Staff	Admin	System rule
Create order	✓	—	—	Server-generated order number; idempotent submit
View own order status	✓	✓	✓	Customer uses order number + contact verification
Accept order	—	✓	✓	Only from NEW
Confirm payment	—	✓	✓	Must record verification actor and evidence
Prepare order	—	✓	✓	Only after payment is accepted
Complete pickup	—	✓	✓	Record pickup timestamp and staff actor
Cancel order	Policy-defined	✓	✓	Allowed stages and refund rules must be explicit
Manage catalog	—	—	✓	Product edits do not mutate historical order snapshots
Manage staff access	—	—	✓	RBAC; audit permission changes

Operational rules to encode in the backend:

- A transition endpoint must validate the current status before changing it; never trust the client-provided previous state.
- Every transition creates an **OrderStatusHistory** record.
- Cancellation is a policy decision, not just another UI button.
- Payment evidence should be stored separately from the order and should support verification status and audit metadata.
- Daily product limits should be checked transactionally against confirmed orders, not only against the current cart.

3. Recommended Model Schema

The schema below is relational and works well with PostgreSQL, MySQL, or another transactional SQL database. It keeps business entities separate from audit/event records.

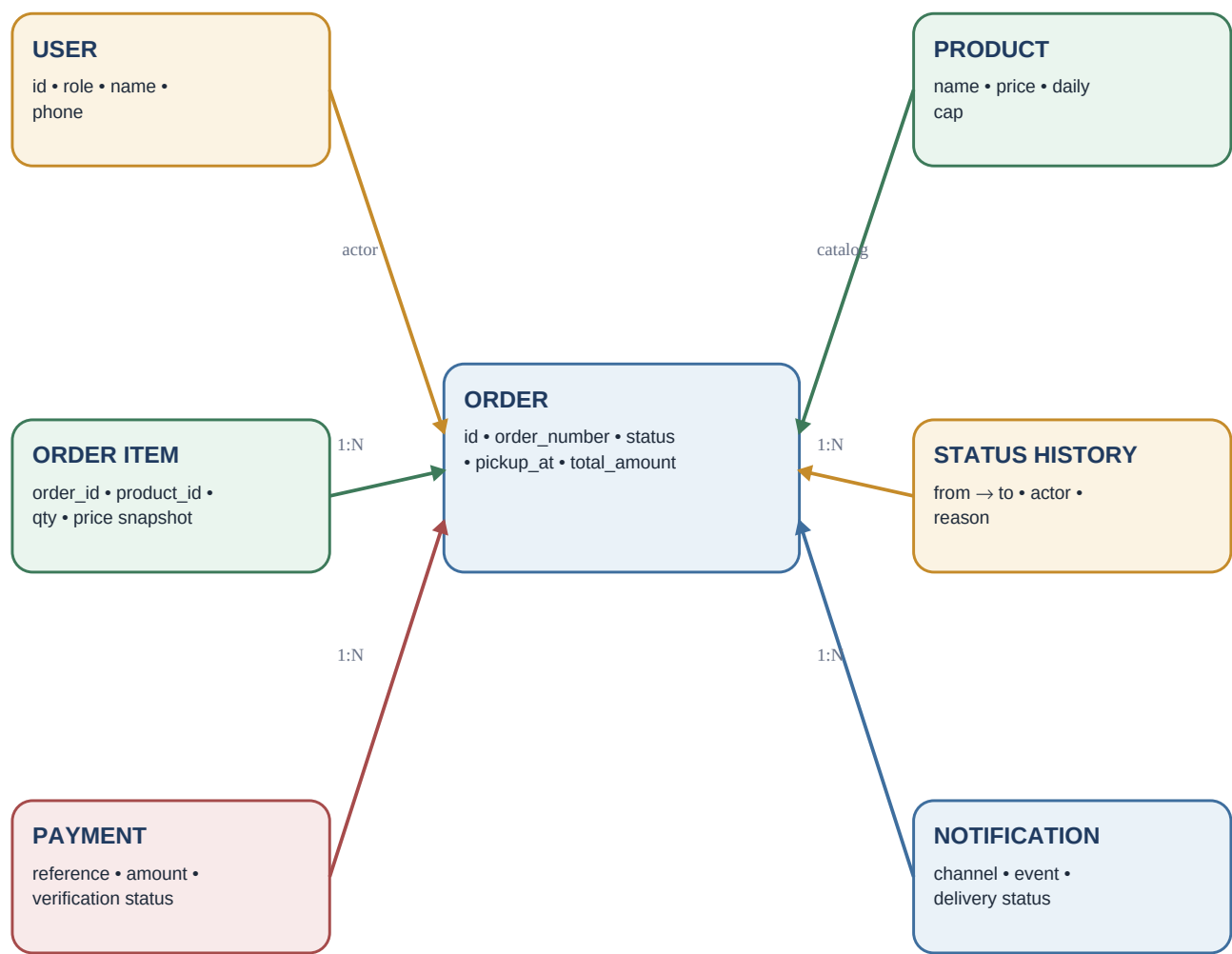
Entity	Key fields	Relationships / purpose
User	id, name, phone, role, is_active, created_at	Represents staff/admin accounts. Customer identity can remain lightweight on the order unless customer accounts are later introduced.
Product	id, name, description, price, max_qty_per_order, max_qty_per_day, is_active	Catalog item. Product image(s) can be modeled separately.
ProductImage	id, product_id, url, sort_order	One-to-many images per product.
PaymentMethod	id, name, instructions, qr_asset, is_active	Admin-managed payment options.
Order	id, order_number, customer_name, customer_phone, pickup_at, status, total_amount, placed_at, completed_at, cancelled_at	Central aggregate. Status is current state; history is stored separately.
OrderItem	id, order_id, product_id, product_name_snapshot, unit_price_snapshot, quantity, line_total	Immutable snapshot preserves historical accuracy after catalog changes.
Payment	id, order_id, payment_method_id, reference_no, amount, paid_at, evidence_url, verification_status, verified_by, verified_at	Supports manual verification now and gateway reconciliation later.
OrderStatusHistory	id, order_id, from_status, to_status, actor_user_id, reason, created_at	Append-only audit trail for operational and admin history.
Notification	id, order_id, channel, event_type, recipient, status, sent_at	Tracks SMS/push/email delivery attempts.
AuditLog	id, actor_user_id, action, entity_type, entity_id, metadata, created_at	Admin/security audit for sensitive changes such as permissions and catalog edits.

Suggested cardinality:

User 1—N OrderStatusHistory; Order 1—N OrderItem; Order 1—N Payment (or 1—1 if the business guarantees one payment record); Order 1—N OrderStatusHistory; Order 1—N Notification; Product 1—N ProductImage; Product 1—N OrderItem.

4. Entity Relationship Diagram

This diagram emphasizes the order aggregate and the auditability of payment and status transitions.



Important implementation detail: the OrderItem should store product name and unit price snapshots. Otherwise, changing a product's name or price later can corrupt historical sales reporting and the admin's sales history.

5. Recommendations & Implementation Priorities

1. Make payment verification explicit and auditable

Use a Payment entity with `verification_status` = PENDING / VERIFIED / REJECTED. Store reference number, amount, timestamp, evidence, verifier, and verification time. If a gateway is introduced later, add `gateway_transaction_id` and reconciliation fields without changing the order workflow.

2. Implement idempotent order creation

Generate the order number server-side and protect submission with an idempotency key or equivalent duplicate-submit guard. This directly addresses double-clicks, refreshes, and order-number collisions.

3. Define cancellation and refund policy before coding

Specify who can cancel, which statuses are cancellable, whether payment is refundable, and how the refund is recorded. A cancellation reason and actor should always be captured.

4. Enforce pickup scheduling rules centrally

Validate business hours, minimum lead time, blackout periods, and optionally daily capacity. The server—not only the UI—must reject invalid pickup times.

5. Treat daily quantity caps as transactional business rules

At checkout/order confirmation, calculate confirmed quantity for the product and date and lock the relevant rows or use a safe counter strategy. The UI may grey out unavailable products, but the backend must be authoritative.

6. Add notifications as an event-driven capability

When status changes to `READY_FOR_PICKUP`, create a notification event. Start with SMS or email and keep the Notification model channel-agnostic so push notifications can be added later.

7. Use RBAC and least privilege

Separate staff operational permissions from admin configuration permissions. Consider permissions such as `ORDER_ACCEPT`, `PAYMENT_VERIFY`, `ORDER_COMPLETE`, `CATALOG_MANAGE`, `REPORT_VIEW`, and `STAFF_MANAGE`.

8. Build reporting from immutable facts

Sales analytics should derive from completed/paid order items and preserve the price snapshot at purchase time. This avoids historical reports changing when catalog prices are edited.

Recommended MVP delivery sequence: (1) catalog + customer checkout, (2) order state machine + staff Kanban, (3) payment verification and audit trail, (4) admin catalog/staff controls, (5) notifications, (6) analytics and operational hardening.

Overall assessment: the concept is straightforward to implement, but the highest-risk areas are payment trust, duplicate submission, cancellation/refund semantics, pickup-time validation, and quantity-cap enforcement. Solving those as explicit domain rules early will make the rest of the UI and API much simpler.